

Project Submission Form

PORTFOLIO & LEADERBOARD

Your certificate proves you showed up. Your project proves you can build. This form turns the project you built in your bootcamp into a published portfolio piece — a permanent link you can put on your CV, share on LinkedIn, and send straight to recruiters.

Top performers in each bootcamp are also featured on the Leaderboard — chosen by instructor grading on the quality of the work. Earned, not voted.

How to use this form

1. Make your own copy of this document (File → Make a copy, or Save As).
2. Fill in every Required field. The Encouraged fields are what make your entry stand out — do as many as you can.
3. Submit by the deadline (Section 6). Instructors review, grade, and publish.
4. Include this completed Project Submission Form inside your final project ZIP.

Read this first — what gets you published (and what gets you rejected)

Every claim must be real. Use numbers from YOUR project — your model's accuracy, your pipeline's throughput, your dataset size. Do not invent statistics like "67% of users prefer...". Made-up numbers get your entry sent back.

No secrets. Ever. No passwords, API keys, tokens, .env contents, or real customer/personal data anywhere in this form or your repo. We check. Anything with a live credential is rejected on sight — and it's a security risk to you.

It must be your own work. The write-up has to match what's actually in your repo. AI-generated filler that doesn't reflect your real project gets rejected. Write it in your own words — recruiters can tell.

1 About You

Full name **REQUIRED**

As you want it shown publicly on your portfolio entry.

Yasith Vimukthi Gunasinghe

Student ID **REQUIRED**

Your Zuu Crew student ID.

1545

Email **REQUIRED**

How we reach you about your submission. Not published.

Yasithvimukthi259@gmail.com

GitHub username **REQUIRED**

e.g. github.com/yourname — links to your public profile.

github.com/yasiith

LinkedIn profile **OPTIONAL**

Full URL. Recommended — this is where recruiters go next.

<https://www.linkedin.com/in/yasithgunasinghe/>

2 Your Project

Bootcamp, track & cohort **REQUIRED**

e.g. "Zero to Data Engineer · Bootcamp 1 · Cohort 2" or "Foundations of ML · Cohort 1".

Zero to Data Engineer Bootcamp 1 Cohort 2

Project title **REQUIRED**

The name of your project or capstone.

ABC Hub: Modern Data Platform with Apache NiFi and PostgreSQL (Mini Project 01)

One-line pitch **REQUIRED**

Max ~20 words. This is the headline recruiters read first. What does it do, for whom? e.g. "A churn-prediction system that streams live events and blocks bad deploys under 75% accuracy."

A NiFi + PostgreSQL pipeline that turns messy streaming-and-rental operational data into trusted, incremental star-schema analytics for ABC Hub.

The problem **REQUIRED**

In 2–4 sentences: what real problem does this solve, and why does it matter?

ABC Hub runs streaming subscriptions and physical rentals on one operational database. Reports took a long time to build, departments reported different numbers, and history was lost because the database only keeps the current state. Streaming and rental behaviour couldn't be analysed together. Management needed one trusted, historical view of customers, content, revenue and inventory without IT hand-building every report.

What you built **REQUIRED**

One paragraph describing what you actually built — the system, the pipeline, the analysis, the model. Plain language.

I built an end-to-end data platform that loads ABC Hub's operational PostgreSQL database (26 tables, 98,700 rows) into a separate analytical database organised as bronze, silver and gold layers.

- **Apache NiFi (74 processors in 6 process groups):**
 - extracts only changed rows using an `updated_at` watermark;
 - keeps every raw row version in bronze for audit;
 - validates mandatory fields;
 - cleans and de-duplicates the data into silver;
 - logs every rejected record.
- **PostgreSQL functions** called by NiFi then build three star schemas in gold: Customer Daily Activity, Content Monthly Performance and a daily Inventory Utilisation snapshot, with SCD Type 2 history on customers and inventory items.
- **Analyst views** give business users a no-joins, no-PII way to query KPIs.
- **Role-based access** controls who sees what.

Tech stack — and WHY you chose each **REQUIRED**

List your main tools (languages, libraries, frameworks, cloud). For the key ones, one line on why you picked it over alternatives. This is what separates a real engineer from someone who copied a tutorial.

- **Apache NiFi 2.12:** required by the brief. It fits well because it gives visual, schedulable flows with built-in failure and retry routing and processor state for incremental extraction. I built the flow as code through NiFi's REST API, so the exported flow JSON is reproducible rather than hand-clicked.
- **PostgreSQL 18:** required as the analytical database. I pushed the aggregations into SQL functions because an incremental customer-day aggregate needs the customer's full history, which sits in the database, not in one NiFi batch.
- **Medallion layers (bronze, silver, gold):** bronze gives audit and replay, silver is the single place data-quality rules live, and gold is fast and business-friendly.
- **Kimball star schema with SCD Type 2:** easier and faster for analysts than joining 26 normalised tables, while keeping history such as "which country was this customer in at the time".
- **NiFi record processors:**
 - `ValidateRecord` checks each record against an Avro schema with mandatory fields.
 - `QueryRecord` cleans the data with SQL, which is readable and generated per table from one config file.
 - `PutDatabaseRecord` with `UPSERT / INSERT_IGNORE` makes re-runs idempotent.
- **Python (standard library only):** builds the NiFi flow and runs it on demand. No extra dependencies to install.
- **Bash + SQL test scripts:** automated validation, idempotency and incremental-load tests, so every result can be re-checked.

3 The Data & The Work

This section is where ML, data and AI projects prove they're real. Fill it in properly — it's the difference between a portfolio entry and a screenshot.

Dataset **REQUIRED**

Where the data came from (source/link), roughly how big (rows/size), and whether it's real, public, or synthetic. If synthetic, say so — that's fine, just be honest.

The ABC Hub operational data package provided by Zuu Crew for this project: a PostgreSQL schema plus 26 CSV files, 98,700 rows covering about 12 months (June 2025 to June 2026). It is **synthetic** course data made to look like real operational data, with deliberately planted quality problems.

Problem type **REQUIRED**

e.g. classification / regression / clustering / ETL pipeline / BI dashboard / RAG / agentic system / streaming inference.

ETL / data warehousing pipeline: incremental batch ETL (NiFi), dimensional modelling (star schemas with SCD2) and an analytics/BI-ready semantic layer.

Your approach **REQUIRED**

How did you actually solve it? Walk through the method — the steps from raw data to result. Bullet points are fine.

- Loaded the source CSVs into PostgreSQL. Some integer columns were exported as "1.0", so I loaded them through a text staging schema and cast them.
- Profiled the data with SQL and found the planted issues:
 - duplicates: 8 customers, 60 streaming sessions, 25 payments;
 - 720 completion rates above 100%;
 - 480 missing watch durations;
 - 170 negative payment amounts and 65 negative late fees;
 - 17 invalid ratings;
 - 12 tickets closed before they were opened;
 - inconsistent text casing.
- Backed each cleaning rule with evidence from the data. For example, negative payments matched the plan fee once the sign was flipped, and missing durations could be derived exactly from start and end times.
- Designed the medallion database: bronze (insert-only history), silver (cleaned current state with a flag on every correction) and gold (7 dimensions and 3 fact tables).
- Built the NiFi flow:
 - incremental extract;
 - raw landing to bronze;
 - validation and cleansing;
 - upsert to silver;
 - error handling with rejected records sent to files and a database table.
- Built the gold loads as PostgreSQL functions, dimensions first and then facts. Each run rebuilds only the customers, months and inventory items that changed since the last watermark.
- Added analyst views, security roles, and automated tests for validation, idempotency and incremental loads.

Results & evaluation **REQUIRED**

The real numbers from YOUR project: accuracy / F1 / RMSE / latency / throughput / rows processed — whatever fits your track. Include what the metric means and, honestly, what the limitations are. No invented numbers.

- **Rows processed:**
 - bronze: 87,676 (all 20 extracted tables);
 - silver: 87,583 after rejecting exactly the 93 duplicate records found in profiling;
 - gold: 68,337 customer-day rows, 7,099 content-month rows and 637,674 inventory item-day rows.
- **Runtime:** the full initial load runs in about 47 seconds on my laptop (12 s extract-to-silver, 34 s gold build).
- **Accuracy:** 44 of 44 automated checks pass. Every fact measure reconciles exactly to the cleaned data, for example 48,000 sessions, 6,500 rentals and 175,581.35 total spend.
- **Idempotency:** re-running the pipeline changes 0 rows, and a forced full rebuild produces byte-identical gold tables.
- **Incremental loads:** a simulated day with 17 source changes rebuilt only 6 of 1,001 customers and 3 months, and 22 of 22 scenario checks passed.
- **Limitations:**
 - The data is synthetic.
 - Rows deleted in the source aren't detected, because that needs change-data-capture.
 - A corrected date on an old event isn't reflected in the monthly content fact until a full refresh.
 - Subscription revenue per title is an allocation based on share of watch time, not a real per-title figure.

4 Proof

GitHub repository **REQUIRED**

Public repo link. Make sure it has a README and no secrets committed.

<https://github.com/yasiith/Z2D-DWDF-Mini-Project-01.git>

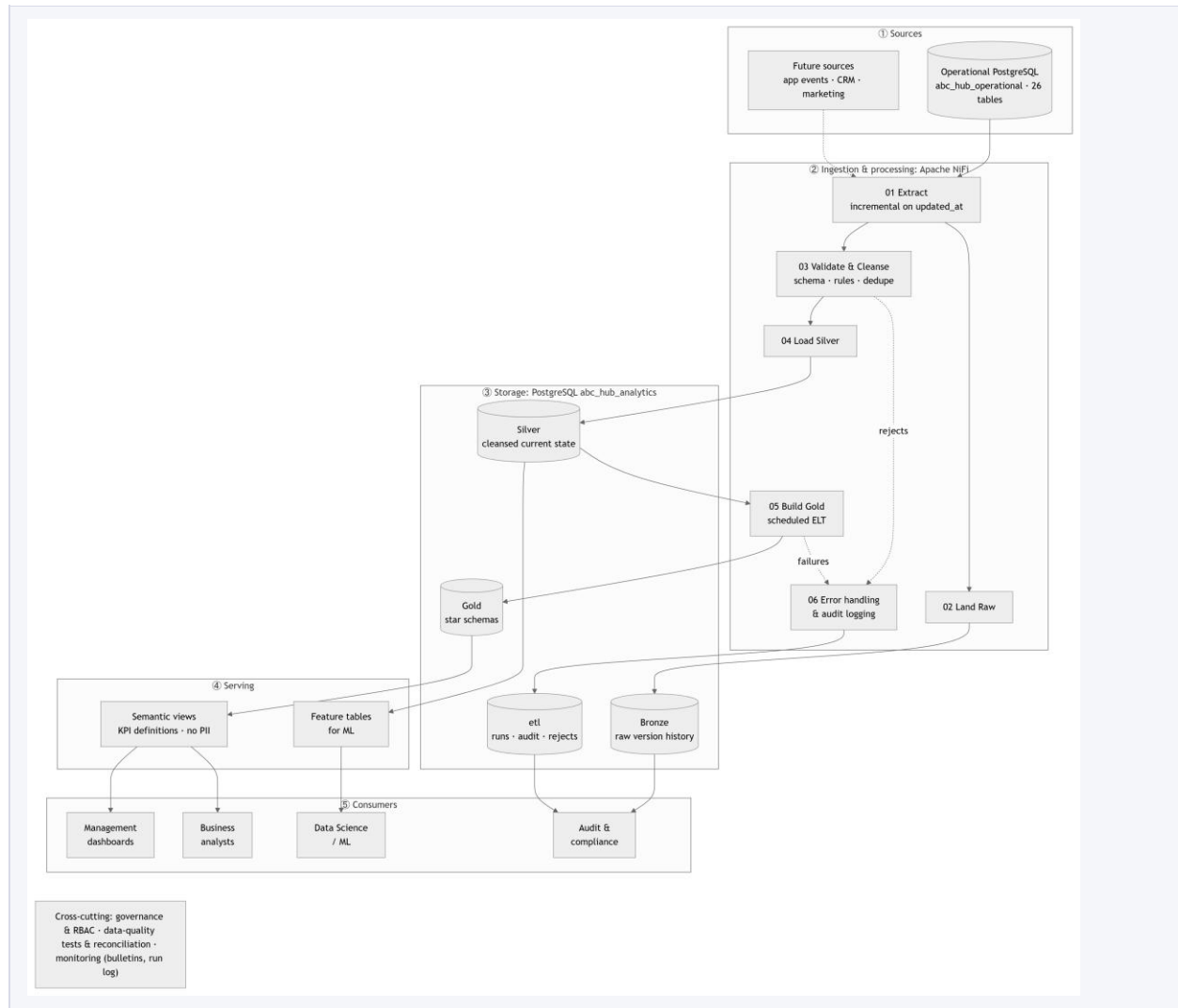
Demo **STRONGLY ENCOURAGED**

The single highest-impact thing you can add. A link to a short video (Loom/YouTube), a live deployed app, or a walkthrough. Paste the link(s).

Type your answer here...

Architecture diagram **STRONGLY ENCOURAGED**

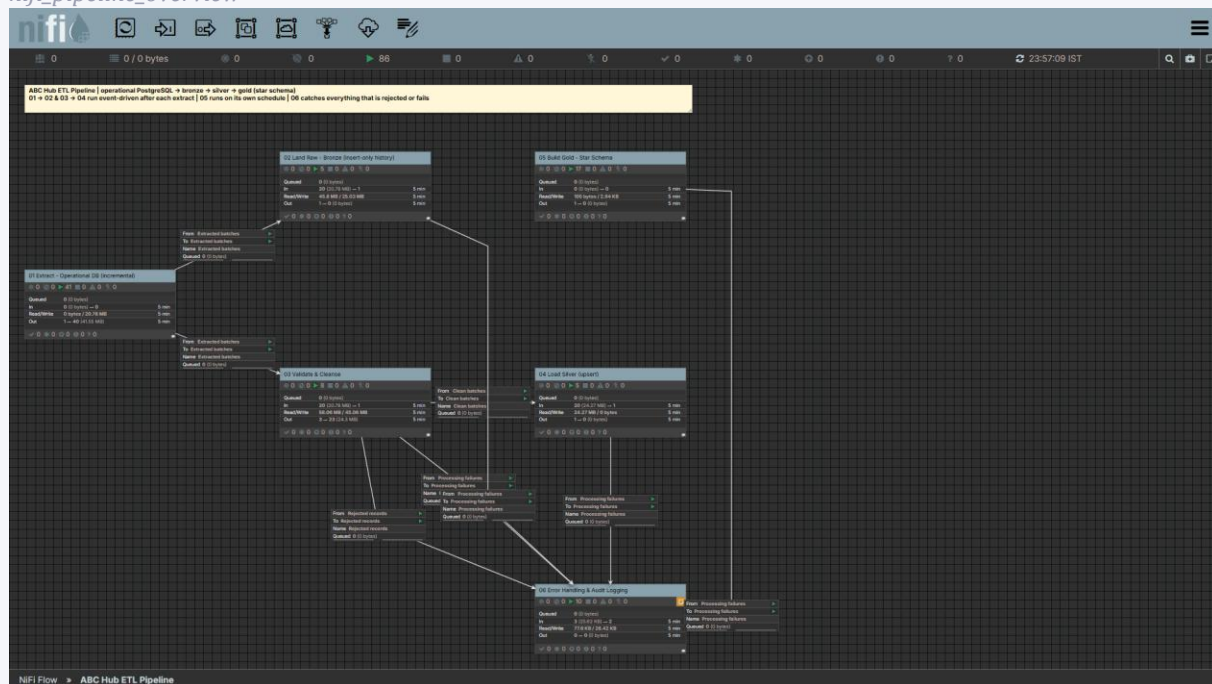
Paste an image, or a link to it (draw.io / Excalidraw / screenshot). Shows you can think in systems, not just scripts.



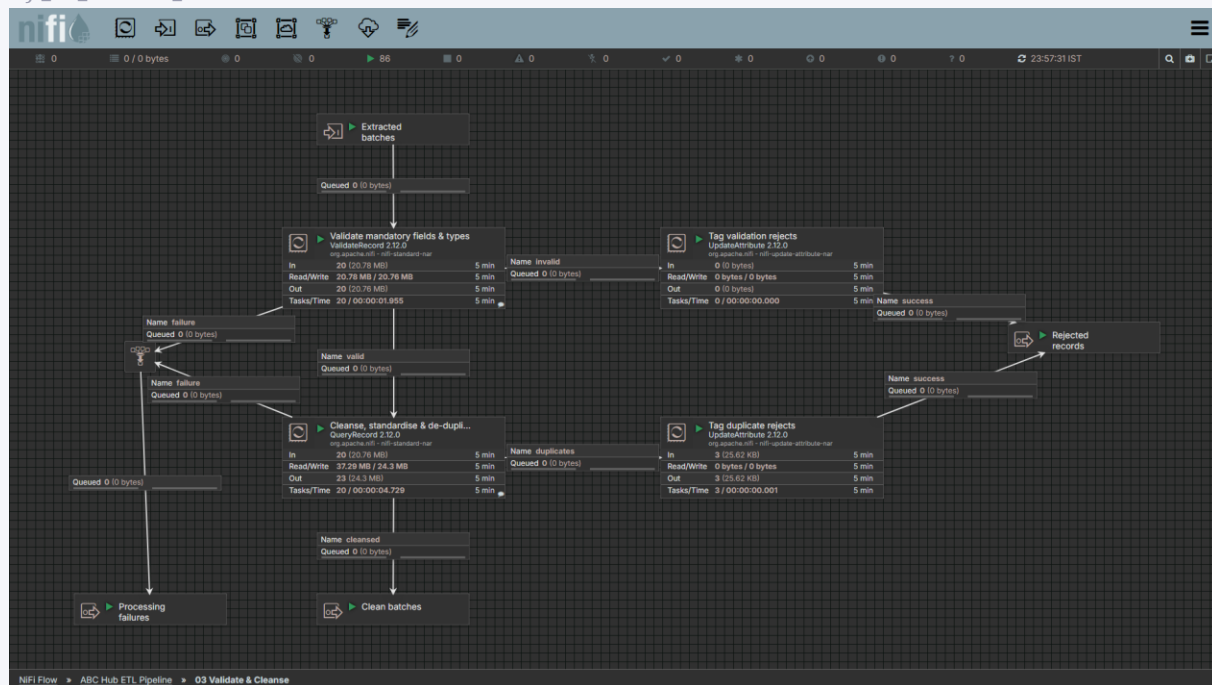
Screenshots / visuals OPTIONAL

Dashboards, plots, UI, sample outputs. Paste images or links — visuals make the entry pop.

nifi_pipeline_overview



nifi_03_validate_cleanse



Successful ETL execution - run log and step log

psql -d abc_hub_analytics - captured 2026-09-24 23:56:58

etl.pipeline_run (gold builds triggered by NiFi)

(4 rows)

etl.run_step_log - initial load triggered by NiFi (dimensions before facts)

(10 rows)

etl.load_audit - batches loaded by NiFi per layer

(2 rows)

ZuuCrew | Project Submission Form

Sample analytical query results

psql -d abc_hub_analytics - captured 2026-09-24 23:56:59

Monthly business KPIs (gold.vw_monthly_business_kpis)

month	active_customers	streaming_hours	rentals	late_returns	subscription_revenue	rental_revenue	total_revenue	refunds	inventory_utilisation_pct	revenue_per_active_customer
2025-06	889	4791.9	441	25	7930.88	10045.01	17975.89	268.49	6.96	20.11
2025-07	932	4876.1	575	83	8144.60	11645.57	19790.17	789.79	10.25	21.33
2025-08	920	4672.0	579	97	8207.52	9113.91	17321.43	288.25	12.67	18.83
2025-09	913	4131.0	552	90	8005.78	8132.77	16138.55	67.82	14.05	17.68
2025-10	897	3881.4	539	88	7615.07	7910.93	15526.00	69.55	15.25	17.31
2025-11	894	3671.2	562	91	7632.11	8007.62	15639.73	186.69	17.09	17.49
2025-12	882	3528.9	552	88	7633.18	7315.76	14948.94	213.01	19.09	16.95
2026-01	882	3441.9	579	86	7414.40	5455.86	12870.26	57.81	21.08	14.59
2026-02	843	3097.3	511	71	7203.63	3930.61	11134.24	49.16	22.49	13.21
2026-03	848	3289.2	524	101	7084.73	3371.40	10456.13	70.91	24.21	12.33
2026-04	846	3295.7	588	106	6869.97	3053.83	9923.80	140.03	25.56	11.73
2026-05	811	3608.1	498	103	6683.18	2065.36	8748.54	102.25	27.02	10.79
2026-06	96	93.8	0	16	5207.67	0.00	5207.67	0.00	23.39	54.25

(13 rows)

Streaming and rental cross-usage by subscription plan

subscription_plan	active_customers	streamers	renters	avg_spend
No active subscription	380	0	309	42.10
Basic	348	348	320	106.30
Standard	305	305	287	159.02
Premium	251	251	238	198.75
Family	93	93	87	258.08

(5 rows)

Warehouse utilisation, May 2026 (top 10)

warehouse_name	warehouse_country	content_type	items_in_service	item_days_rented	item_days_available	utilisation_pct
ABC Hub Warehouse 6	United Kingdom	Video Game	14	197	237	45.39
ABC Hub Warehouse 8	Sri Lanka	Movie	82	970	1572	38.16
ABC Hub Warehouse 9	Canada	Book	33	386	637	37.73
ABC Hub Warehouse 2	Malaysia	Video Game	17	196	331	37.19
ABC Hub Warehouse 1	India	Movie	82	890	1652	35.01
ABC Hub Warehouse 7	United States	Video Game	17	173	327	34.00
ABC Hub Warehouse 10	Australia	Music	44	457	907	33.50
ABC Hub Warehouse 1	India	Music	48	462	1026	31.05
ABC Hub Warehouse 10	Australia	Book	38	365	813	30.88
ABC Hub Warehouse 10	Australia	Video Game	15	144	321	30.87

(10 rows)

14_db_validation_tests

Automated validation suite (sql/03_tests/01_data_validation_tests.sql)

area	test_name	expected	actual	result
silver	no negative payment amounts	0	0	PASS
silver	no negative late fees	0	0	PASS
silver	completion_percentage within 0-100	0	0	PASS
silver	watch_duration never null	0	0	PASS
silver	ratings are 1-5 or null	0	0	PASS
silver	no ticket closed before it opened	0	0	PASS
silver	customer status canonical	0	0	PASS
silver	rental status canonical	0	0	PASS
silver	ticket priority canonical	0	0	PASS
silver	e-mails lower-case and trimmed	0	0	PASS
silver	no duplicate customer e-mails (active rows)	0	0	PASS
silver	every silver row carries a NRI batch id	0	0	PASS
bronze	every silver row has a bronze version	0	0	PASS
bronze	every bronze stream is in silver or in the reject log	0	0	PASS
audit	the 93 profiled source duplicates are all in the reject log	93	93	PASS
audit	no rejected record is active (unflagged) in silver	0	0	PASS
audit	latest gold run succeeded	1	1	PASS
gold	fact A grain unique (customer, day)	68337	68337	PASS
gold	fact B grain unique (content, month)	7099	7099	PASS
gold	fact C grain unique (item, day)	637674	637674	PASS
gold	no fact rows on Unknown (-1) members	0	0	PASS
gold	dim_customer: one current version per customer	0	0	PASS
gold	dim_customer: SCDD versions contiguous (no gaps/overlaps)	0	0	PASS
gold	dim_inventory_item: one current version per item	0	0	PASS
gold	dim_customer rows = active silver customers	1000	1000	PASS
fact A	streaming sessions = silver	48000	48000	PASS
fact A	streaming minutes = silver	2782650	2782650	PASS
fact A	rentals = silver	6500	6500	PASS
fact A	returned items = silver	6055	6055	PASS
fact A	amount spent = completed payments	175581.35	175581.35	PASS
fact A	support tickets = silver	1400	1400	PASS
fact A	wish-list adds = silver	2800	2800	PASS
fact B	streams = silver	48000	48000	PASS
fact B	rentals = silver	6500	6500	PASS
fact B	rental revenue = completed rental payments	80048.63	80048.63	PASS
fact B	allocated subscription revenue = pool (r_j- 1.00 rounding)	95533	95533	PASS
fact B	reviews = silver	3500	3500	PASS
fact B	rating sum = silver valid ratings	13554	13554	PASS
fact B	wish-list adds = silver	2800	2800	PASS
fact C	rentals started in window = silver	6500	6500	PASS
fact C	returns in window = silver	6055	6055	PASS
fact C	rented + available = in service on every row	0	0	PASS
fact C	utilisation within 0-100	0	0	PASS
fact C	every day of the window is present	1	1	PASS

(44 rows)

5 Reflection

Biggest challenge — and how you solved it ENCOURAGED

The single hardest thing you hit, and what you actually did about it. This is often the most impressive part to a recruiter.

The hardest part was making the pipeline both incremental and safe to re-run. NiFi processes data batch by batch, but a correct customer-day or item-day total needs that customer's or item's full history. So I split the work:

- *NiFi handles extraction, validation and cleansing.*
- *PostgreSQL does the aggregation, driven by a "change set" of only the keys that changed since the last successful run. The watermark only moves forward when the whole run succeeds.*

Testing exposed real bugs:

- *Replaying an extract logged rejected records twice, which I fixed by making the reject log idempotent.*
- *My first inventory snapshot took 297 seconds. Rewriting it set-based brought it to 28 seconds, with identical output.*

What you'd build next (V2) ENCOURAGED

If you had another two weeks, what would you add or improve?

- *A Power BI dashboard on top of the analyst views.*
- *Change-data-capture (e.g. Debezium) to catch rows deleted in the source.*
- *A full Data Vault layer between bronze and silver for integrating more sources.*
- *Docker Compose to start NiFi and PostgreSQL with one command.*
- *Alerting (email or Slack) when a gold run fails.*

What you learned OPTIONAL

One or two honest sentences on what this project taught you.

Treat "it ran without errors" as the start of testing, not the end. Reconciliation, idempotency and incremental tests found bugs that a successful run never showed. I also learned when to let the ETL tool do the work and when to push logic down into the database.

6 Consent & Submit

Publishing consent

☐ **I agree** to Zuu Crew publishing this project — my name, project details, and links above — publicly on the Zuu Crew Portfolio and, if selected, the Leaderboard. I confirm the work is my own and contains no secrets or confidential data.

Type your full name to confirm REQUIRED

Acts as your signature.

Yasith Vimukthi Gunasinghe

How to submit

Deadline: [FILL IN — e.g. 20 December 2026, 11:59pm]

Submit to: [FILL IN — submission email or form link]

Important: Submit one final project ZIP. Include this completed Project Submission Form inside the ZIP as a DOCX or PDF, together with your source code, README, assets, and any files required to run or review the project.

Finish your project. That's what gets you on the board.